计算机组成原理 · 补码与加减运算 · 考点精讲

408 各科大题知识点强化课 | 课程第 5 讲（分 P07） | 时长 3 小时 11 分 00 秒

本讲范围：第二章「数据的表示和运算」的前半段——进制转换（快速带过）、**补码的十条性质**（表示范围、求法、特殊补码、相反数、比大小、快速求真值 / 快速求补码、符号扩展）、**补码加减法与三种溢出判断**、**加减运算部件图与四个标志位**；最后两道大题：一道「无符号数与有符号数能否共用同一个加法器」的寄存器代码题，一道补码乘法（含「2n 位乘积只取低 n 位如何判溢出」）。**浮点数（IEEE 754）本讲完全没有讲**——老师只在开头的分值分布里点了它，并明确说下次课也不讲、以后抽空补。

读法建议：先背 §2 的性质表（尤其**性质⑩**）与 §3.2 **三种判溢出**、§3.4 **三个速算套路**——他说这是真题里最吃香的部分；§4 四道题请对着草稿纸跟着算一遍，他的原话是「这种听的话会很乱，可能还是要课下再总结一下」。

一句话：最值钱的两条是**「都是负数时补码可直接当无符号数比大小」**与**「快速求真值：前面一串 F 的负数，加上几能进位成全 0，几就是它的绝对值」**。

目录

1 考情分析：分值、第二章定位与复习优先级

2 核心概念：补码的性质（老师逐条讲的 10 条）

3 解题方法论：加减法、三种判溢出、标志位与速算套路

4 典型题型与模板：四道题完整推演

5 关于「真题」与年份：本讲的边界说明

6 易错点清单

7 考场速查卡

附 本资料的来源与边界

ASR 校正说明（读引用前先看这一段）

转写稿由本地 Whisper 生成，这一讲的错字密度极高，且集中在术语上：**「补码」被写成「补马 / 不马 / 无码」**，**「真值」被写成「帧值 / 增值 / 針織 / 真词」**，**「溢出」被写成「易出 / 一出 / 意出 / 一输」**，**「进位」被写成「净位 / 经位 / 金位」**，**「取反」被写成「区反 / 去反 / 去返」**。本资料引用时统一校正，主要替换：补马 / 不马 / 无码 → **补码**；圆码 / 原马 → **原码**；帧值 / 增值 / 針織 / 真质 / 真词 → **真值**；易出 / 一出 / 意出 / 一输 → **溢出**；净位 / 经位 / 金位 → **进位**；区反 / 去反 / 取返 → **取反**；检法 / 减发 → 减法；地增 / 低升 → 递增；竖轴 → 数轴；净界位 / 进界位 / 敬畏 → 借位 / 进位（CF 的两重含义）；政烈乘法器 / 战略乘法器 → **阵列乘法器**；惩罚 / 成法 → 乘法；乘译 → 乘数；使动作期 / 使用作期 / 始终中期 → 时钟周期；驱子 / 曲指 → 取指；基层器 / 计存器 → 寄存器；敷点数 / 辅典数 / 辅点书 → 浮点数；I triple E 754 → IEEE 754；静置转换 / 禁制转换 → 进制转换；安散 → `unsigned`；无复杂数 / 无福奥数 → 无符号数；有福奥数 / 有发售 → 有符号数；真体 / 整体 / 阵地 / 真词 → 真题。

ASR 校正说明（续）：其余替换词

客下 / 客家 → 课下；技足 → 机组；王道叔 → 王道书；零四体 → 第四题；U-M-U-M-U-L → 无符号乘法（U-MUL）；M-M-U-M-U-L → 有符号乘法（M-MUL）；sab → `Sub`（加法器的减法控制端）；多录选择器 → 多路选择器；简衣 → 简答（⚠ 不确定）。

还有一处要整段存疑：「开始」。讲稿里它反复出现（「重点就是这个*开始*和快表」「下次课直接讲*开始*」「有一道*开始*的题目」），从上下文看是第三章存储系统里与「快表」并列、下次课要专门讲、且大题最多的那个考点，本资料按 **Cache（高速缓冲存储器）**理解，正文出现处均标了 ⚠。**凡不能确定的，一律不写进正文，集中列在文末「来源与边界」。**

1 考情分析：分值、第二章定位与复习优先级

1.1 分值与大题总量（老师口径）

> 「考试的话总共是 **45 分**，其中的话是**十一道选择题**，每道两分，这个是 22；**大题的话是 23 分**。当然这个大题，有时候是 15 加 8，有时候可能是 11 加 12 都不好说，有时候可能 13 加 10，就这个并不是固定的，总共是 45 分。」「408 好歹也是从 09 年到 24 年，总共 16 年。其实 09 到 24，王道书上是只有 **09 到 23 年的真题**，24 年的他没有练，是留到最后当作考前模拟用的。**09 到 23 的话是十五年，每年两道，总共应该是三十道。**」

1.2 大题分布（老师逐个小节点的）

位置（按王道书小节）	大题数	说明
2.2 补码	2 道	本讲主题，「目前是两道」
2.3 浮点数（IEEE 754）	1 道	「考这个一道」——**本讲没有讲**
第三章 存储系统（讲稿作「开始」⚠ 推断为 Cache，与**快表**）	3 + 5 + 2 = 10 道	「重点就是这个开始和快表，这两个经常会一块去考」「第一小节有三道，第二小节有五道」；另有 2 道是「开始」（⚠ 按 Cache 理解）的题被编到别的小节——全书大题最密集处
4.2（指令 / 数据寻址）	4 道	「指令的数据寻址」那几道
汇编	4 道	「会需要有一些汇编语句」；他说「这边其实有一道（Cache）的题目」
数据通路	讲稿未给出数字 ⚠	「数据通路整个组成原理的大题分布相对来说还是比较集中的」（未给数）
流水线	3 道	其中「有一道是 Cache 的题目」
7.3 I/O 方式（中断、DMA 等）	5 道	本章最后一块大题区
合计 30 道		列出的数相加是 29 道，加上未给数的「数据通路」正好 30 道；「整个七章里面只有 8 个小节有大题」

⚠️ 上表是他边翻讲义边连报的数（`[00:01:26]`–`[00:05:36]`），个别数字的归属在转写里已串行。请当**优先级地图**用，不要当精确统计。

1.3 第二章的定位与复习优先级

> 「这个第二章是讲**数据表示和运算**的，这一章其实有点像数学，全是计算，其实和计算机关系并不是特别大，但是它是一个很多其他章节做一些计算题的一个**前置基础**，所以第二章一定要学好。」
> 「第二章的大题虽然并不多，但是**补码相关的选择题很多**，而且在后面的章节的一些计算都会用到这些东西，所以第二章它是经常会掺杂在后面的四五章里面。」
> 「凡是能考大题的知识点，它必定也是有选择题的；凡是能考大题的知识点，往往它这个占的比重比较大。」

内容	老师给的分量	他让做的事
进制转换	复习级	「比较简单了，就简单看一下，如果之前学过的话作为一个复习」；但又强调「要算得又快又准」，因为操作系统分页、组成原理、计网 IP 都天天用
补码的性质与换算	**高频**	本讲主体，「补码有很多性质，很多技巧都是很实用的」
加减运算器与标志位	**高频**	「加加减运算器是要看的，**这些标志位这些都是高频考点**」，还要会算 CF、OF
浮点数（含浮点加减）	本讲未讲	「这个要看，包括浮点数的加减也要看……我们待会也会讲一下」——**实际上整块没有讲**（见文末）
补码乘法	冷门	「到目前为止大纲里面一直有，但是目前的话都是没有考过」
答题时间	老师专门提醒	「考试的话，你选择题 40 个，你要控制在一个小时做完」，「一定要预留充分的时间去做大题」

他点名的题（讲义编号，不是年份）：课后题 29（讲两次）、选择题 33、大题**第四题**（寄存器/代码）、**第五题**（乘法）。

2 核心概念：补码的性质（老师逐条讲的 10 条）

2.1 起点：无符号数与有符号数，同一串机器数两种读法

> 「机器数里面的内容一样……**机器数一样不代表它的真值一样**。你比方说 4 个 1，你无符号数它就表示 15，有符号数的话就表示负 1。」（`[02:26:28]`、`[02:26:35]`）

这是本节所有性质的根：**补码只是「约定的读法」，二进制串本身没有正负。**他最后用这一点解释第四题（同一加法器为什么能同时服务两种数）。

2.2 性质① 表示范围

```
n 位补码的表示范围：  −2^(n−1)  ～  +2^(n−1) − 1

  4 位：  −8  ～  +7          （老师全程用 4 位举例）
  8 位：  −128 ～ +127        （第四题里判 m+n 溢出用的就是这个范围）
 16 位：  −32768 ～ +32767
 32 位：  −2^31 ～ 2^31−1
```

> 「这个表示范围的话，你可以 n 位的话，它的表示范围就是 −2 的 n−1 次方到 2 的 n−1 次方减 1。这个 n−1 是因为要扣掉符号位。」

⚠️ 这句在转写里被叠字成「−2 的 n−1−1−1−1」，上界没念清楚；但 4 位「−8 到 +7」是他后来明确说过的（见 §3.2 方法一）。

2.3 性质② 求补码（正数 / 负数两条路）

> **老师要求背下来的两条**
>
> **正数：**「正数的话是**原码等于补码**」，直接写。
> **负数：**「做的时候一般都是**符号位不变，从右往左，最右边的 1 以及右边的 0 不变，中间的取反**」——他专门说书上那种「按位取反、末位加一」的写法「一般平常做题目都不会这样去用，太慢了」。

```
书上定义：  负数补码 = 原码「符号位外按位取反，末位加 1」
老师口诀：  符号位不动 → 从右往左找到第一个 1 → 这个 1 和它右边的 0 都不动
              → 它左边的各位（不含符号位）全部取反

例（8 位，按他的方向：由补码回原码）
  补码 0x83 = 1 000 0011 → 最右边的 1 及其右边的 0 不动、中间取反
  → 原码 1 111 1101 = −(64+32+16+8+4+1) = −125
  （反过来求 −125 的补码，用同一句口诀，方向相反）
```

他也讲了这个口诀的**反向用法**（由补码回原码）：「补码转原码也是一样，符号位不动，最右边的一不动，中间取反。」

2.4 性质③ 特殊补码（最容易被当成无符号数读错）

```
① 1 后面跟 k 个 0      →  真值 = −2^k          （不是「很大的正数」）
   例：1000 = −8（4 位）、1000 0000 = −128（8 位）

② 全 1                 →  真值 = −1
   例：1111 = −1（4 位）、0xFFFF = −1（16 位）、0xFFFFFFFF = −1（32 位）

③ 前面一串 F、末一两位不是 F  →  用 §2.7 的进位反推法秒出
```

> 「第三个是特殊的补码，如果 1 后面有几个零，**所以这个不要和无符号数搞混了**。如果是无符号数，这里的话应该是一个 16；现在如果是补码的话，那应该是**有几个零就是 2 的几次方**（负的）。还有一个特殊的就是，如果补码全

> 1 的话，它就是负一，真值就是负一。」

2.5 性质⑥ 求相反数的补码（减法转加法的钥匙）

> **一条口诀，方向改成「全都翻」**
>
> 求 `-A` 的补码：**符号位也取反**，然后「最右边的 1 不动、它右边的 0 不动，中间全部取反」，一步到位。

> 「有一个东西它是 A 的补码，你如果要求 A 的相反数，就是 -A 的补码，直接……**符号位也要取反**，最右边的 1 不动，中间的全部要去反，这样就能**一步到位**求出相反数的补码。它的用途就是用来待会去算减法。」
> （`[00:30:07]`–`[00:31:08]`；转写里他先说了半句「符号位不动」随即改口）

⚠️ 上面这段转写里「符号位不动」和「符号位也要取反」连着出现两次（`[00:30:07]`、`[00:30:22]`、`[00:30:38]`），是他自己改口。按他后面的算法与例子（他举的是 +5 与 −5 互为相反数、仅符号位不同），本资料取「**符号位也要取反**」。

> ⚠️ **这里有一处「人算」与「机器算」的差别（本讲后半段专门回来讲）**
>
> 我们背的口诀是**一步到位**：直接写出相反数的补码。
> 而**计算机内部是分两步**：先把减数**全部取反**，再在**最低位给一个进位 1**。老师原话：「我们平常求的时候一步到位了，但是计算机这个加法电路的话，应该是全部先取反，再给一个低位的进位 1。」
> （`[02:02:47]`）

2.6 性质⑨ 补码比大小：把它当无符号数看

> 「这边有一个数轴，下面是它的真值，上面是每一个数的补码……如果都是正数，你可以把补码当成无符号数，刚好是递增的；**如果都是负数，也可以把补码当成无符号数看，也是递增的。**」
> 「-3 是小于 -1 的，然后你会发现它的补码当成无符号数也刚好是小于的……**直接把补码当无符号数**，这是第一个性质。」（`[00:43:20]`–`[00:43:38]`）

第二条性质是「偏移量」：「-3 加上 2 才等于 -1，你会发现它的补码也刚好偏移了一个 2。」他给的用法是：**若甲补码比乙补码（当无符号数）多 2，则甲的真值也比乙的真值大 2。**这在选项挨得很近的选择题里能省一次完整的求真值。

> 性质⑨的两条用法
> (A) 比大小：把补码当无符号数，谁大谁的真值就大（负数之间、正数之间都成立）
> (B) 算偏移：补码（当无符号数）相差 d → 真值也相差 d
> 例（他的）：补码从 8 变到 A（多 2）→ 真值从 −8 变到 −6（也多 2）

2.7 性质⑩ 快速求真值 / 快速求补码（老师说本讲最重要）

> 「这里面其实最重要的性质我觉得就是**第十条**，第十条在很多真题里面能用；第九条用的频率并不高，但是也有。**第十条是用的最多的。**」（`[00:57:42]`）
> 「如果是一个 FFF8，你让它加上一个数，它如果往前进位、后面变成全 0，**加上的这个数就是它的绝对值。**」
> （`[00:47:15]`）

```
【快速求真值】机器数形如「前面一串 F，最后只有一两位不是 F」的负数
   找到最小的正整数 k，使  机器数 + k = 进位成全 0（即 2 的整数次幂）
   则真值 = −k      （k 的十六进制写法就是它的绝对值）

   0xFFF8  →  +8    = 0x10000  → 真值 −8
   0xFFFA  →  +6    = 0x10000  → 真值 −6
   0xFFD3  →  +0x2D = 0x10000  → 真值 −45       （0x2D = 45）
   0xFF96  →  +0x6A = 0x10000  → 真值 −106
   0xFF90  →  +0x70 = 0x10000  → 真值 −112
   0xFFF0  →  +0x10 = 0x10000  → 真值 −16
   0xFFFF  →  +1    = 0x10000  → 真值 −1
   0xF4    →  +0x0C = 0x100    → 真值 −12        （8 位同理）
   0xB0    →  +0x50 = 0x100    → 真值 −80

【快速求补码】就是反过来用：已知真值 −45，要写 16 位 / 32 位补码
   先写出 45 = 0x2D，再想「加上什么会进位」→ 0xD3
   16 位：0xFFD3         32 位：0xFFFFFFD3       （前面补 F，不需要再做取反）
   老师：「这个一定要平时练练，有时候做起来还是挺快的。」
```

> ⚠ **这个方法的适用边界（老师自己划的）**
>
> **「这种方法比较仅限于前面有很多连续的 F、最后只有一两位不是 F」**的机器数。
>
> 他给的反例：像 `0xCAF8` 这种「前面不是 F 的负数，你没办法了，只能先求原码再求真值」；而且「一旦出现这样的数，它的真值会相当大……不太适合考试」。
>
> 另一条经验：「真题里面挺多这种前面全是 F、最后两位或一位不是 F 的数，因为这样的话它的数字其实比较小」，也可能是「负的几十、几百」。

2.8 符号扩展（位数短的负数怎么变长）

> 「如果要扩展成 32 位，符号扩展就先把**符号位保留**，然后**第 15 位也保留**，中间还缺的 16 位**中间全填成符号位**就可以了。」
>
> 「**符号扩展针对的是有符号数**，然后你用这种方式就能求得快一点。」（`[00:53:37]`、`[00:54:57]`）

```
16 位 → 32 位（负数）：高位全部补 1，低位不动
   0xFFD3    （−45）   →  0xFFFFFFD3
   0xFFF7    （ −9）   →  0xFFFFFFF7        ← 第四题附近的 short → int
16 位 → 32 位（正数）：高位全部补 0
   0x007F    （+127）  →  0x0000007F

与「快速求补码」配合：既然是负数，直接按位数把前面写满 F 即可，不必先转二进制。
```

2.9 一张换算表（本讲出现过的数，全部按他的方法核对过）

机器数（十六进制）	二进制	真值	他讲它的场景
0xFFFF / 0xFFFFFFFF	全 1	−1	特殊补码；真题爱出的数
0xFFF8	1111…1000	−8	性质⑩的原例
0xF4	1111 0100	−12	「每四个一组，F4 一眼就是 −12」
0xFF90	1111…0000	−112	快速求真值
0xFFD3	1111…0011	−45	符号扩展与快速求补码
0xFF96	1111…0110	−106	选项 D 的机器数
0xFF9D / 0xFF9E	…1001 1101 / 1110	−99 / −98	选项 B / C，「加 1 真值就加 1」
0xFFDF	1101 1111	−33	x 的机器数（−33 + 0x21 进位）
0x83	1000 0011	−125	「求最小整数」的结果
0x86	1000 0110	−122 / 134	第四题的 x（有符号 / 无符号）
0xF6	1111 0110	−10 / 246	第四题的 y
0x90	1001 0000	−112 / 144	第四题的 z1 = k1
0x7C	0111 1100	+124	第四题的 z2 = k2

3 解题方法论：加减法、三种判溢出、标志位与速算套路

3.1 补码加法与减法的三种写法

```
加法：直接按二进制逐位加，规则只有一条「1+1 得 0 并向前进一位」
      例：+5 + (−2) = 0101 + 1110 = 0011 = +3（无溢出）

减法①：直接相减（可以不做任何转换）
      不够减时「可以认为最高位一定有一个 1」用来借位
      例：+5 − (−2)：0101 − 1110，最低位 1−0=1，中间位不够减就往前借
      他原话：「就算不够减，你从最高位借个一就行了」[00:33:39]

减法②：转成加法 —— 减去一个数，等于加上它的相反数
      A − B = A + (−B)      −B 的补码用 §2.5 的口诀一步写出

两种写法算出来的二进制结果一模一样（他做了两遍对照）。
```

> 「补码的减法是可以**直接减**的，就算不够减，你从最高位借个一就行了。」「无论你是减法直接相减，还是转成加法，它们最后的二进制都是一样的。」（`[00:33:39]`、`[00:38:05]`）

但老师在同一段里立刻加了一道闸门：

> **本讲被他重复最多的一句提醒**
>
> 「**千万不要直接拿结果**，千万一定要判断一下结果有没有溢出。」（`[01:22:25]`）
>
> 「凡是拿机器数去运算，**一定要看一下溢出，因为就算他溢出了，他也会算出一个东西。**」（`[01:33:57]`）

3.2 三种判溢出方法（加法、减法各三种，且互相等价）

```
方法一   按表示范围（先把结果当数学算，再看是否越界）
         例：4 位补码范围 −8 ~ +7；+5 + 3 = +8 越界 → 溢出（上溢）
                                  +5 − (−3) = +8 越界 → 溢出（+8 写成补码是 1000，
                                  而 1000 在 4 位里读作 −8，「不可能 +5 减 −3 得 −8」）
方法二   按符号位（只看参与运算的两个数和结果的符号）
         加法：正 + 正 = 负 → 溢出 ；负 + 负 = 正 → 溢出 ；正 + 负 一定不溢出
         减法：正 − 负 = 负 → 溢出 ；负 − 正 = 正 → 溢出
方法三   按进位（很多同学最熟的那种）
         符号位进位 C_s 与最高数据位进位 C_1 相异或：
         C_s 异或 C_1 = 1 → 溢出 ；= 0 → 不溢出
```

「正常来说其实是三种方式，一共有三种方式：第一种是根据这个表示范围……第二种的话就根据符号位：**正加正等于负，或者负加负等于正，这样的话是溢出；正加负一定是不会溢出**，因为正数加负数肯定不会比原来的正数再大，也不会比原来的负数还小。」（`[00:24:45]`、`[00:26:13]`）

「第三种应该是很多人比较熟悉的一种，就是看最高数据位的进位和符号位的进位……两个一样就没有溢出；0 和 1 不一样，那就溢出了。」（`[00:27:10]`）

「**三种方法都要会，因为这三种方法在不同的场景下，用不同的方法可能会更快一点。**」

书上还有第四种（双符号位），老师怎么说的

「书上也提供了三种，但是和这上面不一样，书上提到了一个**双符号位**，那个其实**用的很少**。」（`[00:28:04]`）——本讲没有展开双符号位的判断规则。

⚠ **机器判溢出 ≠ 人判溢出（讲第四题时他专门停下来强调）**

「人家现在问的是**计算机内部**，你肯定不能说根据表示范围来做，你不能说这个 5 加 3 等于 8、不在这个表示范围里就溢出了，**肯定计算机不是这样做的**——这是我们人在做题的时候可以这样做，计算机在做的时候不会这样去做。」（`[02:29:06]`–`[02:29:22]`）

机器上只能报方法二（符号位）或方法三（进位异或），也就是下面要讲的 `OF`。

3.3 加减运算部件图与四个标志位（书上第 42 页）

「这个**加减运算的部件图是要记得**，这是第一个。」「这个图很万能，通过这一副图，你既能算无符号数的加法和减法，也能算有符号数的加法和减法。」（`[01:45:05]`、`[01:47:56]`）

```
图中的关键部件（★ = 他口述点名过的；X、Y 与选择器作用是按标准结构补的）：
  X、Y          两个操作数输入端
  Sub ★         控制端：Sub = 1 做减法，Sub = 0 做加法
  Y 取反 ★      减法时把 Y 的每一位全部取反
  C_in ★        低位进位：减法时给 1，加法时给 0
  加法器        X + Y(或 Y') + C_in
  C_out ★       加法器最高位的进位输出
  多路选择器 ★  选择结果来源

减法为什么这样接：A − B = A + B' + 1    （取反 + 低位进 1 = 加上相反数）
```

四个标志位（他要求「留意」的）

标志位	含义（老师原话）	公式 / 注意
ZF	「零标志的话，ZF 等于 1 表明结果为 0，**它是无符号数和有符号数都有意义**」	结果全 0 则为 1
OF	「溢出标志的话是**判断有符号数是否溢出**，它是符号位进位和最高数据位进位的异或」	`OF = C_n ⊕ C_(n-1)`；**对无符号数没有意义**（但仍然会被算出来）
SF	「符号标志的话，用来表示结果的符号」	取结果最高位；**对无符号数没有意义**
CF	「进位 / 借位注意他是两个名称」「**加法时 CF 等于 1 表示进位，减法时 CF 等于 1 表示有借位**」	`CF = Sub ⊕ C_out`；**对有符号数没有意义**

> **他点名要记的两个公式**
>
> 「重点留意的就是里面的两个公式，**有的人可能这个标志位不太会算，标志位你直接就用这个公式算**：一个是 OF 的、一个是 CF 的，这两个公式要记一下。」（`[01:47:10]`、`[01:47:21]`）
>
> `OF = C_n ⊕ C_(n-1)`　　`CF = Sub ⊕ C_out`
>
> 其中 `C_n` 是最高位（符号位）产生的进位，`C_(n-1)` 是最高数据位产生的进位。

他用 5 和 2 把四种情形各推了一遍（这是本讲的「母题」）

情形	电路上发生什么	结果	要看的标志位
无符号加法 5+2	Sub=0 → Y 不动、C_in=0，直接相加	7	看 **CF**=0，无进位；OF 照算但无意义
无符号减法 5−2	Sub=1 → Y 取反、C_in=1	3	看 **CF**：`CF = Sub ⊕ C_out = 1 ⊕ 1 = 0` → 无借位（大数减小数，对得上）
有符号加法 (+5)+(+2)	同上，Sub=0	+7	看 **OF**：`C_n ⊕ C_(n-1) = 0 ⊕ 0 = 0` → 无溢出；CF 照算但无意义
有符号减法 (+5)−(+2)	Sub=1 → Y 取反、C_in=1	+3	看 **OF**：`1 ⊕ 1 = 0` → 无溢出；CF 无意义

> 「通过这一个加法器，**无论是无符号数还是有符号数，都是按照这样的规则**：加法的话直接相加，减法的话把减去的那个数取反、低位再给一个进位就可以了。」（`[02:01:21]`）

⚠️ 本讲**全程没有板书可看**（转写里只有他口述的「我这边画一下」「这个叫 C_n，这个叫 C_(n−1)」）。上表把 `C_n` / `C_(n-1)` 标注为「最高位进位 / 最高数据位进位」，是依据他本人的说法与加法器的标准结构还原的；**具体哪根线叫什么，请以教材第 42 页的图为准。**

3.4 三个速算套路（老师反复说真题里最吃香的部分）

套路	什么时候用	怎么做
A. 进位反推绝对值	机器数是「一串 F + 末一两位」的负数	心算「加几能进位到全 0」，加数即绝对值（§2.7）
B. 偏移量法	选项之间只差一两个数（真题选择题的常见布局）	两个补码当无符号数差 d，真值就差 d，不必逐个求真值
C. 直接当无符号数比大小	几个负数比大小	补码（当无符号数）越大 → 真值越大

实战：套路 B——「9D 变到 9E 是加了一个 1，所以立马它的真值就是 −98」：B 选项真值 −99，C 只比它大 1 就是 −98，**不用重算**，「避免你重复计算」。套路 C——比较 3 个 F 开头与 2 个 F 开头的两个负数，「当作无符号数去比，明显这个更大一点」。

> **速算的代价：老师自己划的风险**
>
> 「像我们讲的这种快速做法，如果你平常练得太少，可能就熟练度不够，考试容易卡壳……**最原始的那个方法它虽然慢，但是往往不会出错。**」（`[01:17:16]`）
> 他的建议：**每种题练到能用两种以上方法做，再用另一种当检验**——「不同的方法算出来结果肯定是一样的」。

4 典型题型与模板：四道题完整推演

4.1 【模板】给补码求最小 / 最大整数（课后题 29 第一处）

```
他的做法（原话）：
「你想让他求最小的，你肯定让这个 1……首先是最小的整数应该是负的，这个符号位应该占一个 1；
 你想让他最小，你让这个 1 越往后靠就行了。」[01:01:50]–[01:02:02]

板演的结果与三种算法（同一个答案，速度差很多）：

 补码 = 1000 0011 = 0x83

 算法①（最慢，求原码再算）：符号位不动、最右的 1 不动、中间取反
        原码 = 1111 1101 = −(64+32+16+8+4+1) = −125
 算法②（快一点）：原码右边那段看成「7 个 1 减 2」= 127 − 2 = 125 → −125
 算法③（他推荐的，一步）：0x83 加什么会进位？8 + 7 = 15 进位 → 0x7D
        → 绝对值 0x7D = 7×16 + 13 = 125 → 真值 −125

 他还延伸讲了同族小问：
 最大整数 → 符号位取 0，后面的 1 越靠前越好
 最大负整数 → 符号位取 1，1 越靠前越好（补码当无符号数越大，真值越大）
```

⚠ 老师**只念了做法和结果，没有念题面**（位数、以及「为什么不是 0x80 = −128」这类约束在讲稿里都没有出现）。上面 0x83 是他自己写出来、并说「前面是一个 8、后面是一个 3」的那个数，本资料只还原他有把握的

部分，题面请以讲义为准。

4.2 选择题 33：补码直接比大小

```
做法：都是负数 → 当真无符号数比大小 → 3 个 F 开头的那个最大 → 选 D

顺手算真值（用套路 A）：
  某个选项 0xFFDF → 加 0x21 进位 → 真值 −33      （0x21 = 33）
  另一个 0x...F   → 加 0x21 进位 → 真值 −33
「真题经常会给你这种特殊的例子」
```

4.3 x − y 的选项题（老师口中的「29 题」第二处）：三种方法互相检验

```
已知（他念出的机器数）：x = 0xFFDF = −33       y = 0x0041 = +65
求：哪一个选项等于 x − y

【方法一 转真值】
  −33 − 65 = −98   → 找哪个选项的真值是 −98
  用套路 A/B 核对选项：
    B 选项 0xFF9D → −99
    C 选项 0xFF9E → −98   ← 比 B 大 1（偏移量法），答案
    D 选项 0xFF96 → 加 0x6A 进位 → −106

【方法二 直接相减】
  0xFFDF − 0x0041 = 0xFF9E
  「这个 F 减 1 是 E，15 减 1 是 14，第 13 减 4 是一个 9，前面剩下的全是 F」
  ★ 关键：不能直接拿结果！要判溢出 —— 负数减正数 = 负数，符号没变 → 无溢出 → 结果可用

【方法三 转成加法】
  减去 65 → 加上 −65；−65 的补码：0x41 → 加 0xBF 进位 → 0xFFBF
  0xFFDF + 0xFFBF = 0x1FF9E → 取 16 位 = 0xFF9E
  判溢出：负 + 负 = 负 → 无溢出

三种方法都得到 0xFF9E（= −98），互相验证。
```

他在这一题上花的时间最多，理由是他自己说的：「这个题目要用三种方法去做，因为要用这三种方法，刚好会把我们讲过的都复习一下。」

4.4 真值法的局限 + 符号扩展（x = 127、y 是 short）

```
题目给的条件（转写里只念出一部分）：x 与 y 相加，问哪个选项正确

他先用的「快办法」：x = 127（0x7F），y = −9 → 127 − 9 = 118，直接找 118 的选项；
   再看符号：结果是正数 → 排除为负的选项 → 只剩 D

他随即讲这个办法的局限：「如果考试给的 x 和 y 他不是很小的数，你这种方法就不太好用了；
   或者给的 x、y 不是真值，给的就是一些二进制数或者十六进制数……这种方法其实不太适用。」

于是改用机器数 + 符号扩展（y 是 short 类型，16 位 → 32 位）：
   y = 0xFFF7（−9）  ——符号扩展——>  0xFFFFFFF7
   x = 0x0000007F（+127）
   相加：低四位 F + 7 = 22 → 写 6 进 1 …… 最终 0x00000076 = 118

   判溢出：正 + 负，一定不会溢出 → 结果有效 → 选 D
「一定要判断有没有溢出，因为就算他溢出了，他也会算出一个东西。」
```

⚠️ 这一题的题面（x、y 的声明类型与取值范围）在转写里没有完整念出，「0x7F / 0xFFF7 / 0x00000076」这几个数是他写的；**题干细节请以讲义为准。**

4.5 大题·第四题：无符号数与有符号数能不能共用一个加法器

题干要点（他念的）：一段代码，8 个寄存器保存若干变量；**带 `unsigned` 的是无符号数，没带的是有符号数**（「有符号数的话是用补码」）；问 `r1`、`r5`、`r6` 的内容用十六进制表示是多少。

```
变量与寄存器（按他念的）：
   r1 = x（无符号）       r2 = y（无符号）
   r3 = m（有符号）       r4 = n（有符号）
   r5 = z1 = x − y        r6 = z2 = x + y
   r7 = k1 = m − n        r8 = k2 = m + n

数值：
   x = 134 = 0x86          y = 246 = 0xF6       （255 − 9 = 246，「八个一减一个九」）

第（1）问  r1 = 0x86                  （0x86 = 128 + 4 + 2 = 134；讲稿念的是「128 加 6 加 4 加 2」，
与 0x86 不符，按 0x86 订正）
          r5 = x − y：0x86 − 0xF6 → 0x90
          r6 = x + y：0x86 + 0xF6 = 0x17C → 取低 8 位 = 0x7C
          ★「不用管他溢不溢出，计算机无论溢不溢出他都会把低位的给保留，
             除非题目里明确问你有没有溢出。」[02:15:59]
```

```
第（2）问  m 与 k1 的真值
          m 的机器数还是 0x86（无符号数赋给有符号数，机器数不变）
          → 快速求真值：0x86 + 0x7A 进位 → 真值 −122    （0x7A = 122）
          n 的机器数还是 0xF6 → 0xF6 + 0x0A 进位 → 真值 −10
          k1 = m − n = −122 − (−10) = −112
          ——也可以走机器数：k1 的机器数 = r5 的机器数 = 0x90 → 0x90 = −112（一致）
```

第四题真正要你察觉的那件事（他说这是本题的核心）

「**R1 和 R3 的机器数是一模一样的，R2 和 R4 的内容是一模一样的；同理 5 和 7 一样、6 和 8 相等。**」「**只要加法器两端的输入一样，你计算完他的输出就是一样的。**」「无符号数相减和有符号数相减，这个过程加法器算完的过程都是一样的……**无非是解释方式不一样**。」（`[02:26:19]`、`[02:22:57]`、`[02:33:49]`）

他给的课下练法：「把代码的每一项**全部用机器数来做**，求出每个机器数之后你去关注一下这里面每一个机器数有什么区别，就能分析出刚才的这些东西。」

第（3）问　有符号数加减和无符号数加减能否用同一个加法器实现？

答：可以。理由（按他的说法组织）：

① 两者的机器数送进加法器的两个输入端；

② 加法就直接相加，减法就把第二个加数**全部取反**、再给低位**一个进位 1**；

③ 算出一个机器数之后，是无符号数就按无符号数解释，是有符号数就按有符号数解释。

第（4）问　有符号数运算是否溢出、哪一个溢出？

m + n = −122 + (−10) = −132

8 位有符号数只能表示 −128 ~ +127 → **m + n 溢出**

机器数验证：0x86 + 0xF6 = 0x7C，负 + 负 = 正 → 溢出

m − n = −112，不溢出（负 − 负 = 负）

★ 计算机内部的判法只能回答：① 根据符号位（正+正=负 / 负+负=正）

② 根据符号位进位与最高数据位进位（异或）

他的评价：「单纯说做出这个题目应该还是能做的，**但是一定要察觉出这一点**。」

4.6 大题·第五题：补码乘法（四问）

第（1）问　ALU 只能做加减与逻辑运算，没有乘法指令，只有加法、减法和移位指令，能否实现乘法？

答：可以，用**加法 + 移位**。

他举的十进制例子：26 × 35（展开式「26×(3×10) + 26×5」是整理时补的算式）。

二进制例子：13 × 6 = 13 × (4 + 2) = (13 左移 2 位) + (13 左移 1 位)

「就是把这个东西给他拆成多项，拆成多项之后每一项相乘就变成移位，移完之后再相加。」

⚠ 他还举了 1101 乘「1 + 4 + 8 + 32 + 64」（转写把 64 写成「624」），说明要累加好几项；那一段的位数念得乱，只保留结论：**累加次数取决于乘数。**

第（2）问　若已有乘法指令（ALU + 移位器 + 寄存器 + 控制逻辑），控制逻辑的作用是什么？

答：**控制累加的次数**（用循环实现）。

原话：「这个到底要加多少次，就是通过一个循环来控制的……控制逻辑主要是用来保证这个循环的次数，用来表示把多少个相加、多少个累加起来，其实这个次数就取决于乘数那个数的位数。」[02:47:58]

第（3）问　三种实现方式，哪一种时间最短？
① 没有乘法指令（要用多条加法 + 移位指令）　→ 最长
② 有专门的乘法指令（只有一条指令，但一条指令要消耗多个时钟周期）→ 居中
③ 用阵列乘法器　→ 最短
他补充：① 里的加法指令、移位指令「它也是一条指令，凡是一条指令在执行的时候都要经过取指」，
而取指本身「一般至少会消耗三个时钟周期」。
⚠ 他描述取指三步的微操作被转写成「放到 MAL / MPL / MDL、R」，无法辨读，只保留结论。
关于 ③：「阵列乘法器整个能保证在一个时钟周期完成」——他说这是「按照答案的说法」，
「书上也没有写」，「比较冷门，就不用深究」。

第（4）问　乘积能保留 2n 位，现在只取低 n 位，判断溢出
★ 前提结论：「你把两个 n 位的相乘，最后用 2n 位一定不会溢出。」
他的类比：x1 < 10000、x2 < 10000，两个相乘一定小于 10^8，用 8 位一定能表示。
例子（他说题目给的）：x·y = 2^32 − 2
2^32 − 2 = 0xFFFFFFFE，写成 64 位就是 0x00000000FFFFFFFE（高位补 0）
无符号数：32 位最大能表示 2^32 − 1，比 2^32 − 2 大 → **没溢出**
有符号数：32 位最大只能表示 2^31 − 1，比 2^32 − 2 小 → **溢出**

★ 只取低 n 位时怎么判溢出（他分无符号 / 有符号讲了两遍）：
无符号：看高 n 位是否全 0
全 0 → 没溢出（丢掉的高位全是 0，不影响数值）
非全 0 → 溢出（丢掉的部分里有有效数字）
有符号：看高 n 位（连符号位一起看，32 位时他说「高 33 位」）
正数：高 n 位全 0、符号位为 0 → 没溢出（符号位变 1 会把正数读成负数）
负数：高 n 位全 1、符号位为 1 → 没溢出（丢掉的 1 还原成原码是 0，不改数值）
他的类比：8 位补码强行只留低 4 位，中间那些位若是符号位的重复就不影响数值。

「这个高 33 位都是一的话就不会溢出，全零的话也没（溢出），但凡不满足这两点都会溢出。」（`[03:07:42]`）
「**后面有一道真题问了**，所以就放在一块去看一下。」（他补了一句：问的是有符号数怎么判溢出；`[03:07:54]`）

5 关于「真题」与年份：本讲的边界说明

年份认定依据（照实说）

老师在本讲 3 小时 11 分里，没有说出任何一道题的具体年份。真正的年份只有两处，说的都不是某道题（另有一处 `[00:03:15]`「就放到别的年份了」，他是把「小节」说成了「年份」）：
① `[00:01:56]`「408 好歹也是从 09 年到 24 年，总共 16 年。其实 09 到 24，王道书上是只有 09 到 23 年的真题，24 年的……留到最后当作考前模拟用的」——这是**复习资料的范围**，不是题目年份；
② `[00:02:22]`「09 到 23 的话是十五年，每年两道，总共应该是三十道」——同上。
因此，本资料**没有给任何一道题标注年份。**

他明确称之为「真题」的，只有下面这几处（都是「这类考点真题里常出」，不是具体某年）：

位置	他的话（校正后）
`[00:51:34]`	性质⑩快速求真值「**对于真题来说特别好用**」
`[00:57:42]`	「第十条在很多**真题**里面能用；第九条用的频率并不高，但是也有」
`[01:15:14]`	「尤其这个最后一条快速求真值，你看一下，这种**真题特别喜欢出现这种数字**」（一串 F 开头、末一两位不是 F）
`[02:41:29]`	「就补码的乘法，这个其实**到目前为止大纲里面一直有，但是目前的话都是没有考过**」
`[03:07:54]`	「但是**后面有一道真题问了**，所以就放在一块去看一下」（⚠️ 他说问的是有符号乘法判溢出）
`[00:58:36]`	「先通过题目把我们刚才讲的用一下，**真题都有的东西才会拿出来讲**」

他给的题号（**29、33、第四题、第五题**）是讲义 / 课后题编号；其中「29」念到两次（`[00:58:52]` 是「求最小整数」，`[01:09:33]` 是 x−y 的选项题）⚠️ 是否同号不同问、或转写把「39」听成「29」，讲稿无法判断，按他念的照录。

本资料里凡属教材补充的地方（例如「−2^(n−1) 没有对应的原码表示」「原码 1000 0000 是 −0」）都会在句中直接标注「教材补充」，不混进他的口径。

6 易错点清单

错误 / 坑	老师的说法与正确做法
把补码当无符号数读	「**这个不要和无符号数搞混了**」：`0xFFFF` 在 16 位补码里是 **−1**，不是 65535；`1000` 是 −8，不是 8
「1 后面一堆 0」当成大正数	补码里 1 后面 k 个 0 = **−2^k**
不判溢出就直接用结果	「**千万不要直接拿结果，千万一定要判断一下结果有没有溢出**」「就算他溢出了，他也会算出一个东西」
以为计算机也按「表示范围」判溢出	那是人做题的方法；机器只能按**符号位**或**进位异或**（OF）判
混淆 CF 与 OF	**无符号数看 CF，有符号数看 OF**；CF 在加法里叫进位、减法里叫借位；OF 对无符号无意义、CF 对有符号无意义（但它们仍会被算出来）
用「正+负」去担心溢出	「**正加负一定不会溢出**」——比原来的正数不会再大、比原来的负数也不会再小
以为机器数相同就是同一个数	「**机器数一样不代表它的真值一样**」：4 个 1，无符号是 15、有符号是 −1
符号扩展补 0	有符号数扩展时中间要**补符号位**（负数补 1）：`0xFFF7 → 0xFFFFFFF7`；正数才补 0
要写 −45 的补码时先写原码	不用：「既然它是负数，直接按位数把前面写满 F，末两位写 0xD3」——**一步到位**
把「一步到位」当成机器内部的算法	人算可以一步到位；**机器是「全部取反 + 低位进 1」两步**
速算法用在不该用的数上	进位反推只适合「前面一串 F」的数；`0xCAF8` 这种只能老实求原码。练不熟还不如用原始方法
算完减法不做检验；乘法溢出判据记混	减法他建议每题用两种方法（直接减 / 转加法）互检，「不同方法算出来的结果肯定是一样的」。乘法：2n 位乘积**一定不溢出**，只有**只取低 n 位**时才要判——无符号看高 n 位是否全 0，有符号看高 n 位是否全为符号位的重复

7 考场速查卡

【一】补码基本功
n 位范围：−2^(n−1) ~ 2^(n−1)−1　　（4 位：−8~+7；8 位：−128~+127）
正数：补码 = 原码
负数求补码：符号位不动，最右边的 1 及其右边的 0 不动，左边各位取反
负数补码回原码：同样一句口诀
特殊值：1 后 k 个 0 = −2^k ; 全 1 = −1 ; 0x80 = 最小负数 −128
相反数 −A 的补码：符号位也取反，最右边的 1 及其右边的 0 不动，中间取反

【二】两条速算（老师说是真题里用得最多的）
进位反推求真值：0xFFF8 + 8 → 进位 → −8 ；0xFFD3 + 0x2D → −45 ；0xFF96 + 0x6A → −106
偏移量比较：两个补码当无符号数差 d，真值就差 d（选项挨得近时用）
比大小：都是负数时，补码当无符号数，越大真值越大

【三】加减法与判溢出
加法：1+1 得 0 进 1 ；减法：① 直接减（不够减就从「最高位那个 1」借）② 转成加法 A+(−B)
判溢出三法：① 表示范围（人算快） ② 符号位（正+正=负 / 负+负=正；正+负必不溢出）
　　③ 符号位进位 ⊕ 最高数据位进位 = 1
减法判溢出：正 − 负 = 负 → 溢出；负 − 正 = 正 → 溢出；或转成加法用上面三法
★ 机器内部只认 ② ③，对应 OF

【四】加减运算部件图（书 P42）
Sub = 1 减法 / Sub = 0 加法；减法：Y 全取反 + C_in = 1
ZF = 1 表示结果为 0（有无符号都有意义）　　SF：结果符号（无符号无意义）
OF = C_n ⊕ C_(n−1)　（有符号溢出）　　CF = Sub ⊕ C_out （加法=进位，减法=借位）
有符号看 OF，无符号看 CF

【五】乘法（第五题）
没有乘法指令 → 加法 + 移位（13 × 6 = 13<<2 + 13<<1）
控制逻辑 → 控制累加次数（循环次数），取决于乘数的位数
时间：阵列乘法器（1 个时钟周期） < 乘法指令（多周期） < 加法+移位多条指令
2n 位乘积一定不溢出；只取低 n 位才判溢出：
　无符号：高 n 位全 0 → 不溢出
　有符号：高 n 位全为符号位重复（正数全 0 / 负数全 1）→ 不溢出；否则溢出

【六】符号扩展
16 位 → 32 位：保留符号位与第 15 位，中间全部填符号位
　0xFFD3 → 0xFFFFFFD3 ；0xFFF7 → 0xFFFFFFF7 ；正数补 0

如果只记五件事

① **负数求补码的一句口诀**（符号位不动、最右的 1 及右边的 0 不动、左边取反），反向也用它；
② **性质⑩快速求真值**：一串 F 开头的负数，加几进位到全 0，几就是绝对值——他说这是「用得最多」的一条；
③ **三种判溢出**，并且知道**机器只认符号位与进位异或**，还知道「正+负一定不溢出」；
④ **OF = C_n ⊕ C_(n−1)、CF = Sub ⊕ C_out**，有符号看 OF、无符号看 CF；
⑤ **加法器不区分有无符号**——输入一样输出就一样，区别只在解释方式。

附：本资料的来源与边界

来源	内容	可靠性与边界
B 站《408 各科大题知识点强化课》第 5 讲「计算机组成原理 · 补码」，分 P07，音频 **3 小时 11 分 00 秒**	§1–§7 的全部	本地 Whisper 转写。讲稿 `.md` 共 2565 行 / 约 11.9 万字节，时间戳覆盖 `[00:00:05]`–`[03:10:47]`，即**全程**；其中 `[01:35:15]`–`[01:44:18]` 是课间休息（约 9 分钟无内容）。错字密度极高，引用处已统一校正并集中列在文首的 ASR 校正框。**本讲全程没有板书可看**（转写里只有「先把东西我先画一下」），所有公式、数轴、电路图与寄存器数值都是老师口述的，整理时按他念出的运算顺序还原。
教材通识（标注为「教材补充」的地方）	①「−2^(n−1) 没有对应的原码表示（原码 1000 0000 是 −0）」；②双符号位（变形补码）判溢出的规则	**不是老师讲的。**他在 `[00:28:04]` 只说了一句「书上提到了一个双符号位，那个其实用的很少」，没有展开；「−128 没有原码」这一点他也没有讲。这两处若要写进答案，请以教材为准。
整理时的算术补全	第四题的 0x90 / 0x7C 与 8 位范围核对；2^32−2 = 0xFFFFFFFE 的 64 位写法	0x90、0x7C 是他念出的；0xFFFE 那一处属掉字还原（见 ⑪）。其余只补了中间进位/换算过程，不涉及新知识点。

> ⚠ **转写含糊、按上下文推断的地方（逐条列出）**
>
> ① `[00:30:07]`–`[00:30:50]` 讲「求相反数的补码」时，他先说「符号位不动」、紧接着改口「符号位也要取反」，转写里两种说法都在。本资料按他随后的例子（+5 与 −2 只差符号位）取**符号位也要取反**（§2.5）。
>
> ② `[01:01:50]`「求最小整数」只念了做法（「让这个 1 越往后靠」）与结果 `0x83 = −125`，**题面约束没有念**：为什么不是 0x80（−128）讲稿里没有信息，本资料未做推断（§4.1）。
>
> ③ `[01:07:10]`「再来回忆一下 33 题」与 `[01:09:33]`「看一二十九题」两处编号，可能有一处是「39」的误听；本资料按他念的照录（§5）。
>
> ④ `[01:09:33]`–`[01:26:22]` 那道「x−y 选项题」的题面只念出 `x = 0xFFDF`、`y = 0x0041` 与四个选项的部分机器数，**完整题干与选项 A 的内容没有出现**（§4.3）。
>
> ⑤ `[01:29:11]`「你直接 127 减去 9 是一个 118」一段，x、y 的声明类型与题面在转写里残缺（只有「这个 y 是 short 类型」这一句是清楚的）；本资料只保留 127、0xFFF7、0x00000076 = 118 这条链（§4.4）。
>
> ⑥ `[01:45:05]` 之后的加减运算部件图：`C_n` / `C_(n−1)` / `C_out` / `C_in` 的对应关系是我按他的说法与标准结构还原的；**具体连线以教材 P42 为准**（§3.3）。
>
> ⑦ `[02:10:42]`「这个 x 就是这个 130」，随后他又说「这是 134」并全程使用 `0x86`。本资料取 **x = 134 = 0x86**（0x86 与 134 自洽），⚠ 原音究竟是 130 还是 134 无法确证（§4.5）。

⚠ 不确定清单（续）

⑧ `[02:16:38]`「134-216 不够减的」：按他前面算出的 y = 0xF6 = 246，此处应为「134 减 246」；本资料按 246 处理；同段他把 8 位范围说成「−128 到 −127」，本资料按 −128~+127 写（§4.5）。

⑨ `[02:46:43]` 他把乘数念成「1 加 4 加 8 加 32 加这个 624」，其中「624」按全文互证应为 **64**（与「乘 6 相当左移六位」一致）；本资料按 1+4+8+32+64 理解，只保留「累加次数取决于乘数」的结论（§4.6）。

⑩ `[02:49:29]` 取指三步的微操作被转写成「放到 MAL / MPL / MDL / R」，无法辨读，本资料只保留「取指一般至少三个时钟周期」这一结论（§4.6）。

⑪ `[02:58:30]`「转成十六进制应该是 FFF,F,F0,F1」是掉字，按 2^32−2 = `0xFFFFFFFE` 还原（§4.6）。

⑫ `[03:03:29]` 起讲的「高 32 位 / 第 32 位 / 高 33 位」三种说法混用；按其结论（正数全 0、负数全 1）归一为「高 n 位为符号位的重复，32 位时即高 33 位相同」（§4.6）。

⑬ 讲稿里反复出现的「开始」（`[00:02:43]` 等），本资料按上下文理解为 **Cache**，⚠ 未得到讲稿直接确认，正文出现处均已标注（§1.2）。

⑭ `[00:03:45]`–`[00:04:10]` 的「简衣」「开始的屏幕」两处无法辨读（疑为「简答」「Cache 的题目」），本资料未采信，§1.2 表中以 ⚠ 标出。

老师没讲完 / 本讲没有覆盖的内容

① **浮点数（IEEE 754）整块没有讲**：他只在开头把它列进了大题分布（2.3，一道），中途提过一句「这个要看，包括浮点数的加减也要看」，但**本讲完全没有进入浮点数**——没有阶码、尾数、规格化、IEEE 754 格式的任何内容。课尾他明确说：「下一次上课我们就浮点数就先不讲，直接讲（Cache），因为下次如果讲浮点数（再讲 Cache）可能时间就不太够了……浮点数那边还有一道大题，到时候抽空的话再补一下。」（`[03:09:20]`–`[03:09:34]`）

② **反码与移码**：本讲从头到尾**没有系统讲反码和移码**（「反」只作为「取反」这个动作出现）。因此本资料没有「原码 / 反码 / 补码三者互转」的对照表，只写了老师实际讲的原码 ↔ 补码。

③ **双符号位判溢出**：他只说书上提过、用得很少，没有展开。

④ **补码乘法的具体步骤**：他明确说「这种东西并不需要你专门去学那种乘法的步骤，就了解这个过程就可以了」，本讲只讲了第（1）–（4）问的方法论。

老师没讲完 / 本讲没有覆盖（续）

⑤ **第三章与之后的部分**（Cache、指令、数据通路、流水线、I/O）：只在 §1.2 的分布表里出现，本讲没有进入。

⑥ **他给的部分课后题没有讲**：本讲只细讲了 29、33、第四题、第五题，其余按他的说法是「课下自己做一下」。

⑦ **课间休息**（`[01:35:15]`–`[01:44:18]`）无授课内容；课尾 `[03:08:46]` 之后是对第二章两句大题的总体评价与下次课安排，与考点无关，未收录。